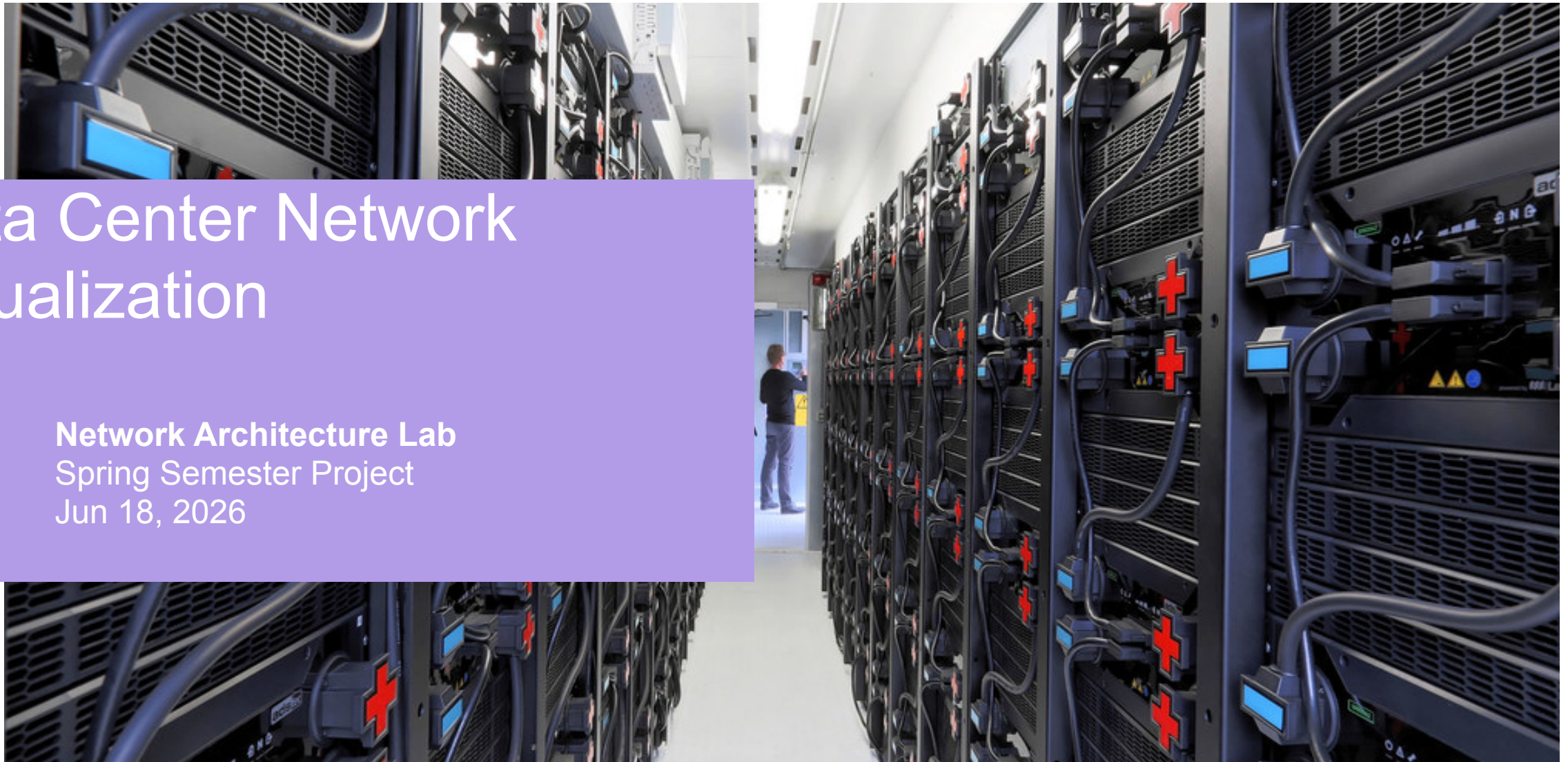


# Data Center Network Visualization

**Network Architecture Lab**  
Spring Semester Project  
Jun 18, 2026



# Agenda

1. Motivation
2. System Architecture
3. Implementation Details
4. Demo
5. Analysis
6. Challenges
7. Future Work

# Why Data Center Network?

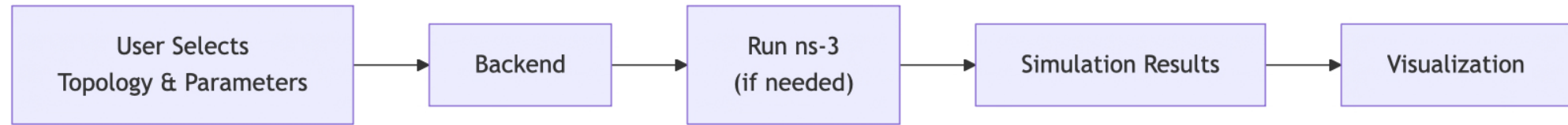
## Motivation

- Data center networks are an active research area.
  - New congestion control algorithms
  - New topologies and configurations
  - Limited access to production data centers
- Researchers rely heavily on simulation.
  - Simulations generate large amounts of data
  - Understanding network behavior remains difficult.
- **We need a more intuitive visualization tool !**
  - Real-time understanding of network dynamics
  - Modular and adaptable to different scenarios
- It provides **intuition to answer questions** such as "What changes when we switch congestion control algorithms?"

```
[packet-log] t=1.093ms link=0-16 event=rx id=1 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=1-16 event=rx id=16 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=2-17 event=rx id=31 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=3-17 event=rx id=46 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=4-18 event=rx id=61 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=5-18 event=rx id=76 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=6-19 event=rx id=91 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=7-19 event=rx id=106 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=8-20 event=rx id=121 size=368 queueDelay=0.046ms linkDelay=1.046ms
[packet-log] t=1.093ms link=9-20 event=rx id=136 size=368 queueDelay=0.046ms linkDelay=1.046ms
```



# System Architecture

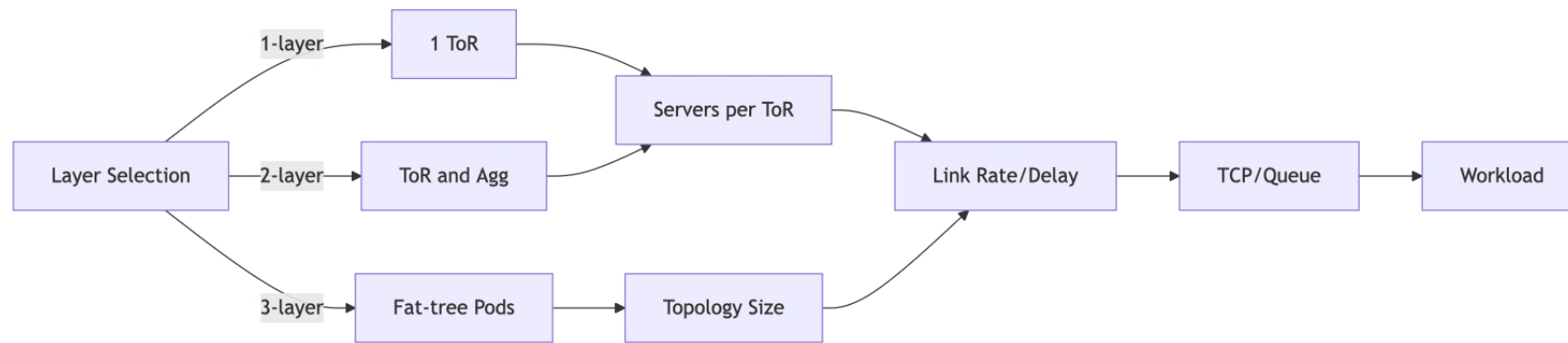


## Implementation details

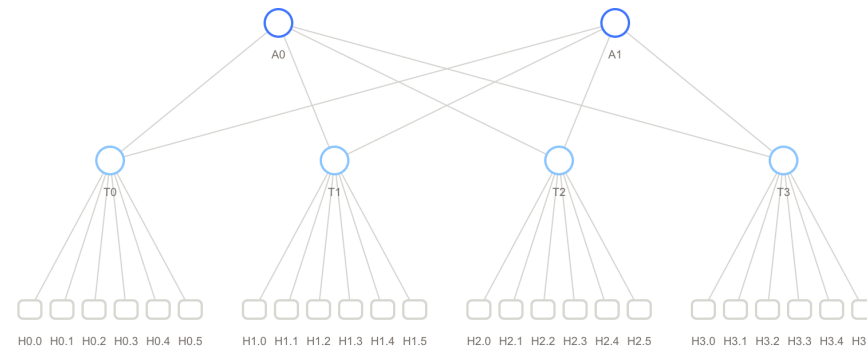
- Frontend: React, Next.js, Typescript
- Backend: FastAPI
- Simulation: **ns-3**, C++
- Separated components to support future extensions to other network scenarios

# Implementation Details

- Topology Configuration Wizard

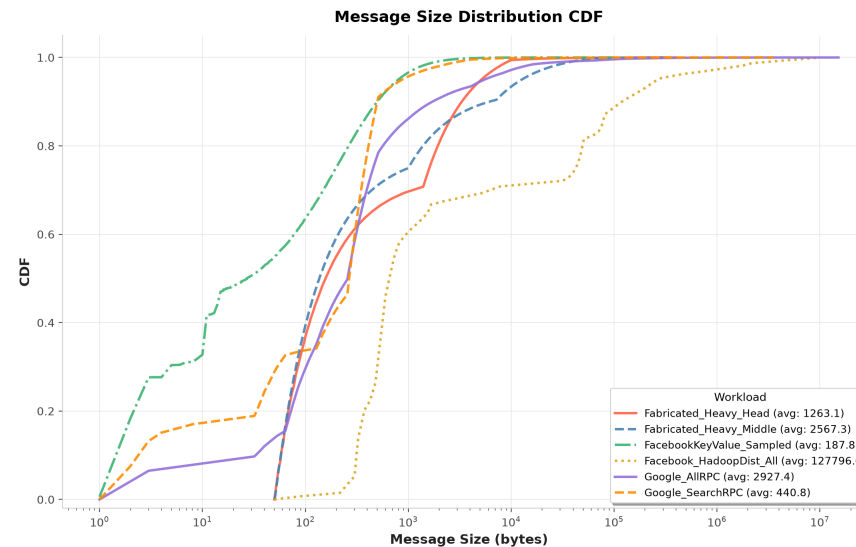


- Topology Generation



# Implementation Details

- Traffic Generation



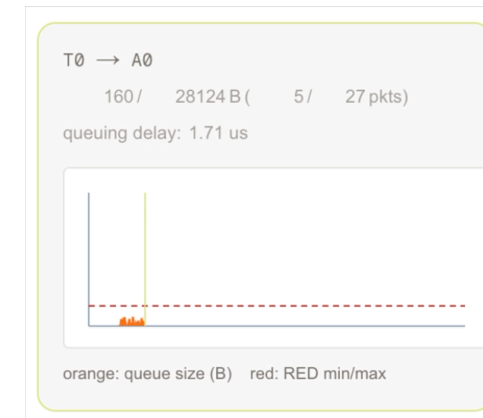
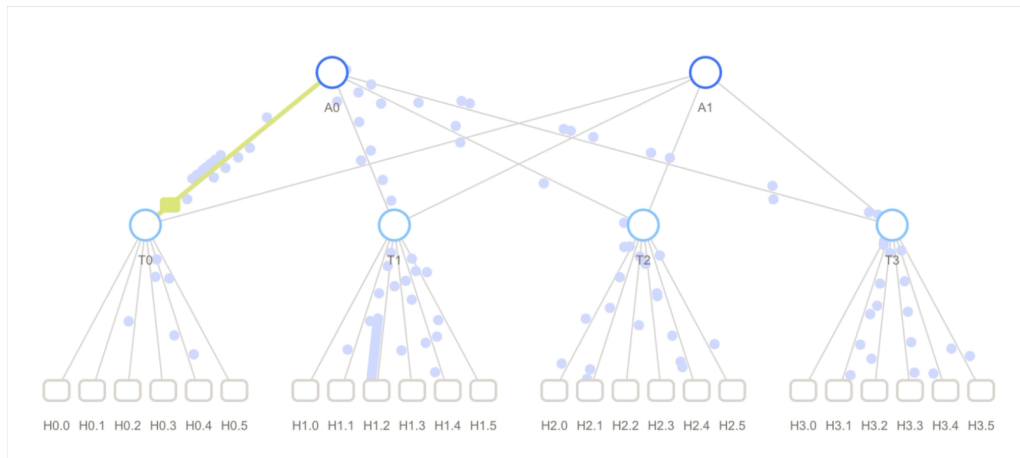
- Independent Poisson process per source
- Target load + average message size  $\rightarrow$  arrival rate
- Exponentially distributed inter-arrival times
- Message size sampled from workload CDF
- For each arrival, the destination selected uniformly among other hosts

# Implementation Details

- Queue Instrumentation and Data Collection

```
id,size,enqueue_time,dequeue_time,arrive_time
1,36,0,928,51856
2,36,0,1856,52784
3,36,0,2784,53712
4,36,0,3712,54640
```

- Frontend Visualization

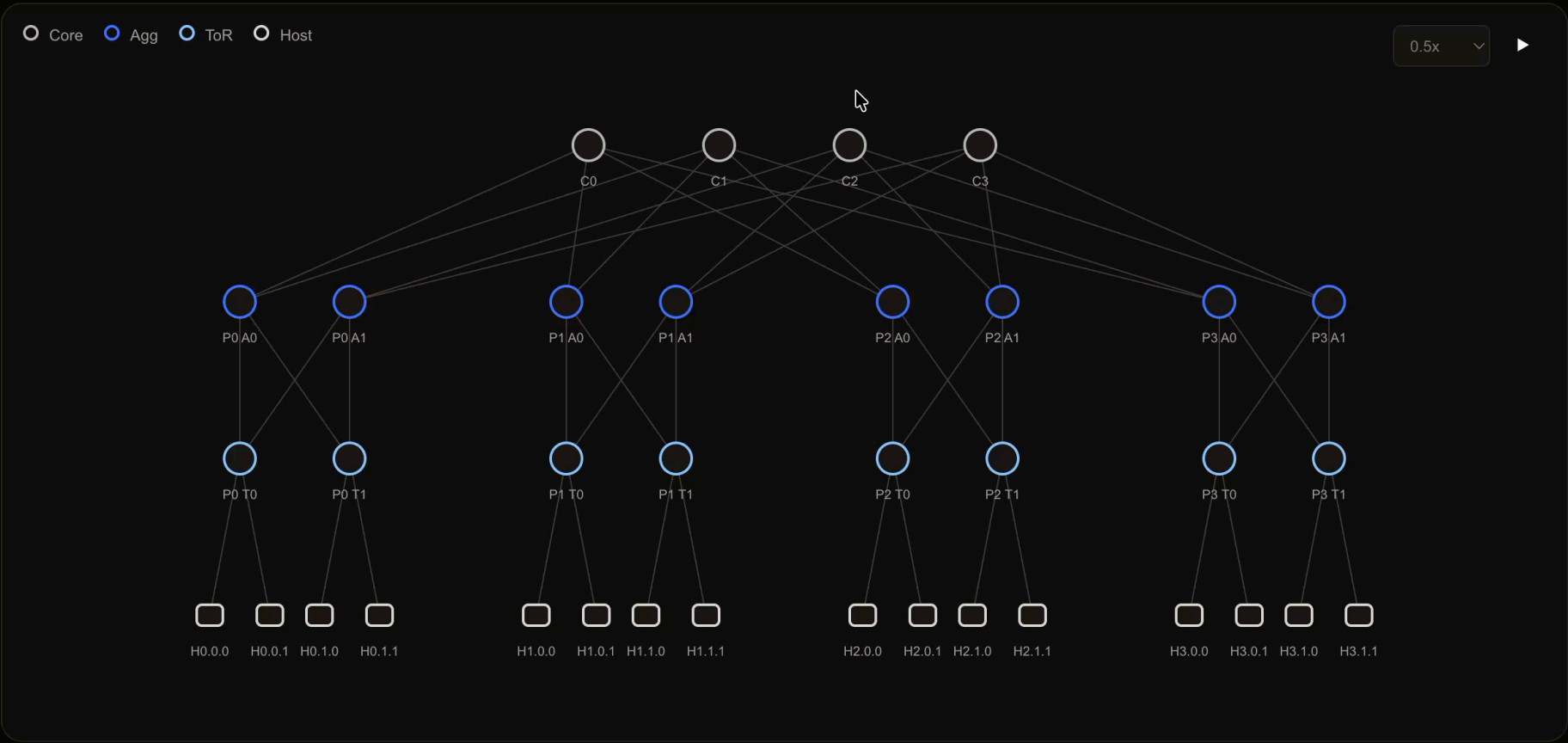


# DCN Visualization

3-layer 4 pods S-ToR: 10Gbps ToR OS: 1:1 ToR-Agg: 10Gbps Agg-Core: 10Gbps Delay: 1ms TcpNewReno  
FifoQueueDisc Load: 50% Google\_AllRPC

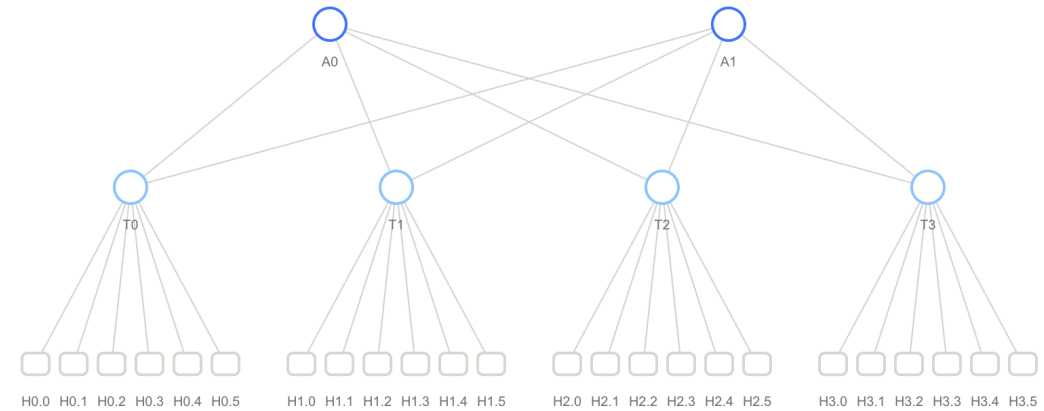
Configure

Run



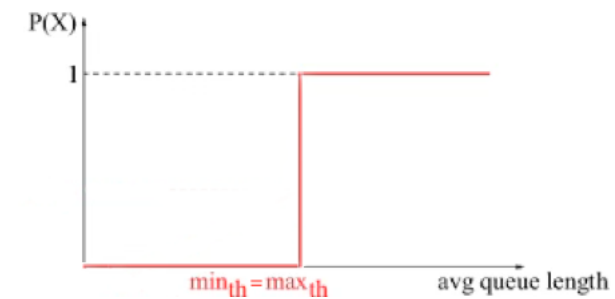
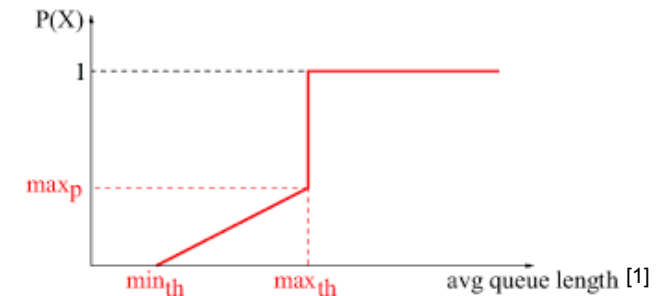
# Analysis

# Analysis



## Parameters

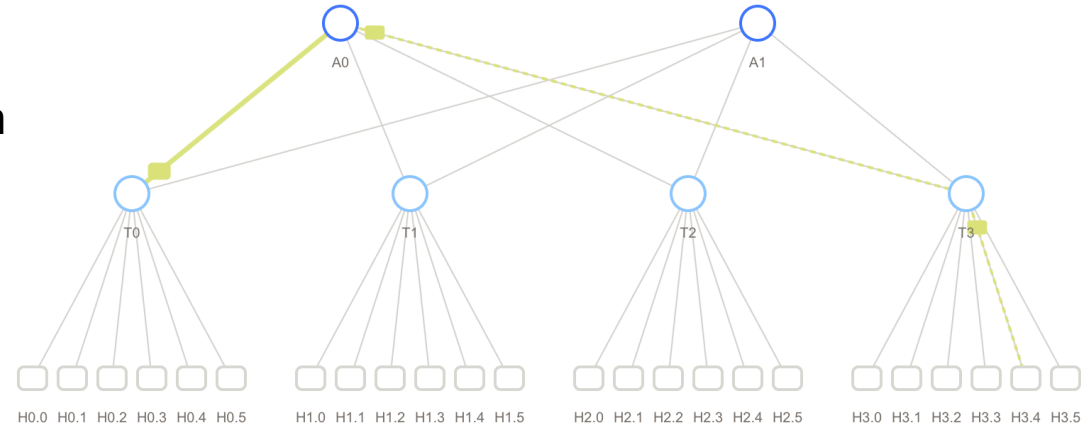
|                                       |                                                   |
|---------------------------------------|---------------------------------------------------|
| Server to ToR Link Rate               | 500Mbps                                           |
| ToR Oversubscription Ratio            | 2                                                 |
| ToR to Agg Link Rate (calculated)     | 750Mbps                                           |
| Link Delay                            | 50us                                              |
| Congestion Control Algorithm          | TCPDCTCP                                          |
| Queue Algorithm                       | REDQUEUE                                          |
| <b>RED Min/Max Thresholds</b>         | <b>{ 15, 25, 45 }</b>                             |
| Load                                  | 95%                                               |
| <b>Message Distribution Workloads</b> | <b>{ Google_AllRPC, Facebook_HadoopDist_All }</b> |



1. Y. Feng et al., "Managing Internet Routers Congested Links with a Kohonen-RED Queue", 2001.  
The graph below is adapted from [1].

# Analysis

## RED Thresholds Comparison

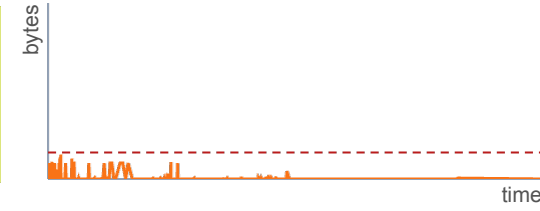
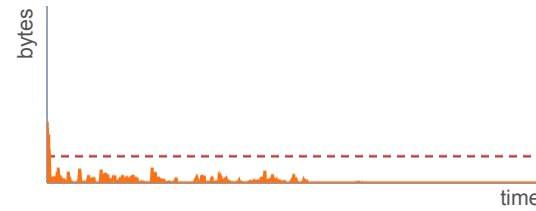
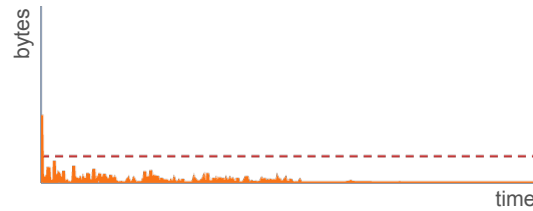


T0 → A0

A0 → T3

T3 → H3.4

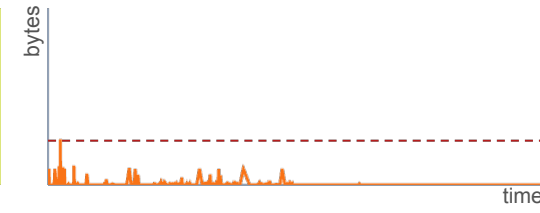
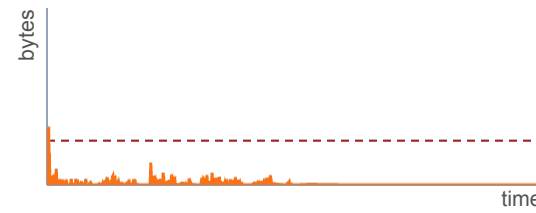
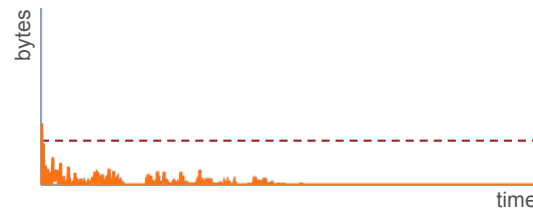
RED min/max: 15%



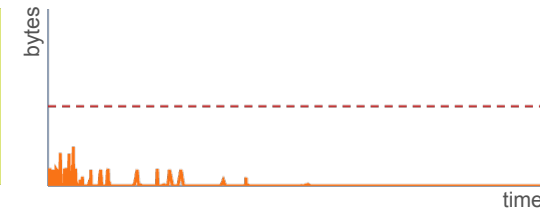
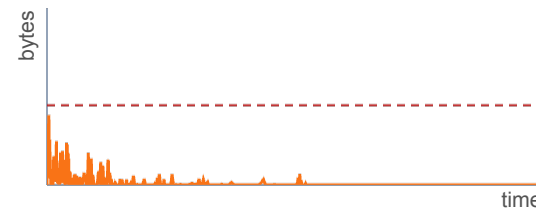
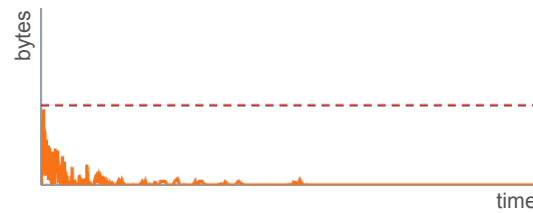
orange: queue size  
red: RED min/max thresholds

Google\_AIIRPC

25%



45%



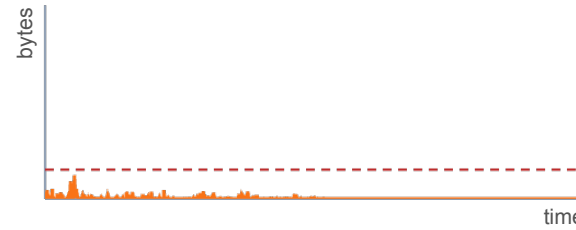
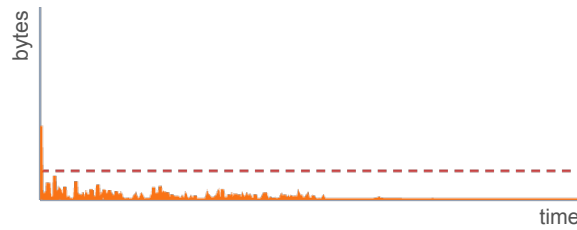
# Analysis

## Workloads Comparison

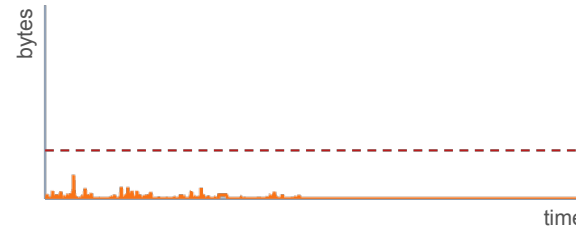
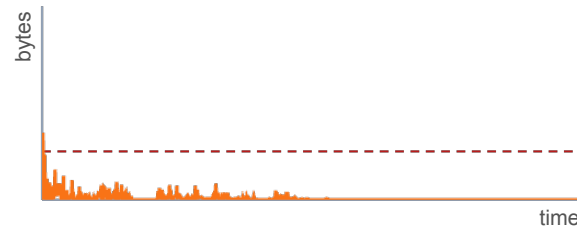
Google\_AIIRPC

Facebook\_HadoopDist\_All

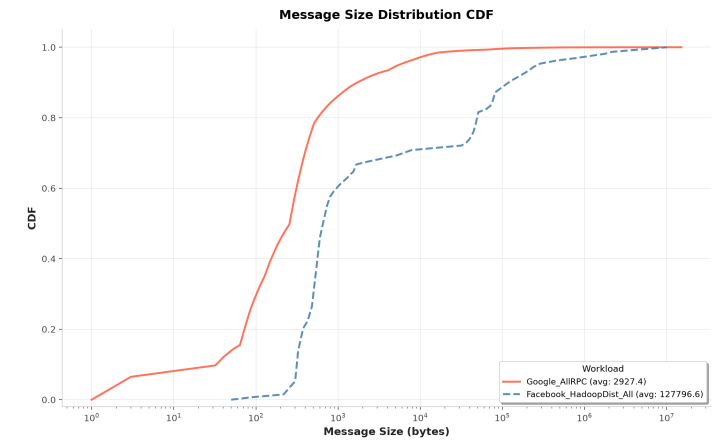
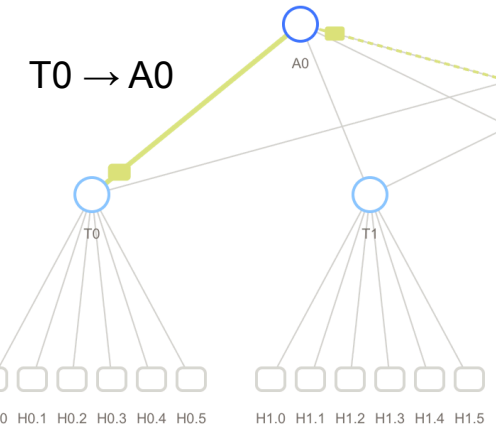
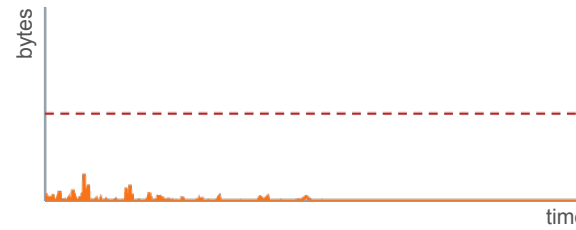
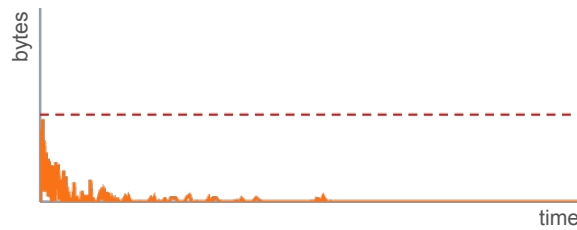
RED min/max: 15%



25%



45%



# Challenges

```
Config::SetDefault( "ns3::Ipv4GlobalRouting::RandomEcmpRouting", BooleanValue(true));
```

## Challenge 1: ns-3 TCP Assertion Failure

- **Problem**

- Simulation crashed under specific DCN configurations
- Failure occurred when ECMP and TCP DCTCP were used together

- **Root Cause**

- ECMP introduced packet reordering
- Reordered packets generated unexpected duplicate ACKs
- Triggered an internal ns-3 TCP assertion

- **Solution**

- Investigated ns-3 source code and community discussions [1]
- Disabled ECMP for this project configuration

1. TCP Duplicate Ack in CA\_OPEN state  
<https://groups.google.com/g/ns-3-users/c/fit3blP4CY>

# Challenges

## Challenge 2: Scalability and Visualization Stability

- **Problem**

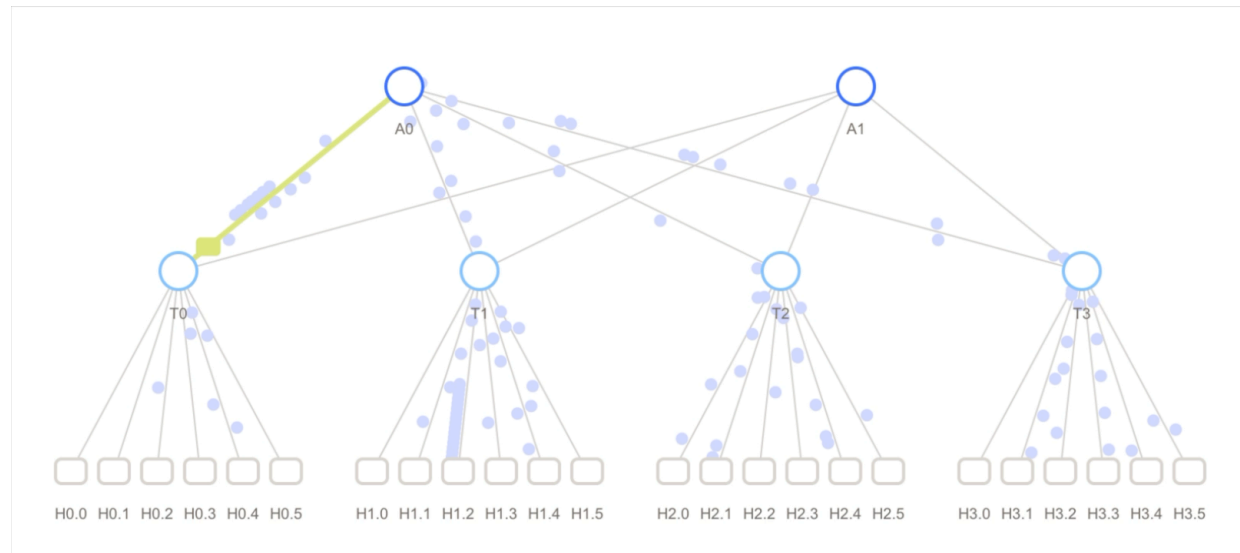
- Too many events produced in the simulation
- The frontend visualization became overloaded
- The UI started to freeze or crash, especially in high-rate scenarios

- **Solution**

- Simulation duration was scaled with RTT
- Precompute queue event series, use snapshot lookup, and downsample graph points
- Use per-link event cursors and limit rendered packets dots under dense traffic

# Future Work

- Support additional network topologies
- Simulation and Visualization can be done in real-time



# Thank you

**Sam Lee**

EPFL

Jun 18, 2026

# Appendix

## Error Log

```
DCN layers=2 k=4 tor=4 agg=2 serversPerTor=6 hosts=24 topoTor=4 topoAgg=2 core=0 serverToTorRate=500Mbps  
torToAggRate=750Mbps aggToCoreRate=750Mbps loadPct=80 workload=Google_AllRPC maxHostHops=4  
maxRttSeconds=0.0008 simTime=1 queueBytes=49999 redMinThresholdPct=15 redMaxThresholdPct=15  
queueDiscImpl=ns3::RedQueueDisc links=32  
NS_ASSERT failed, cond="m_dupAckCount == 1", msg="From OPEN->DISORDER but with 4 dup ACKs", +0.010322730s  
0 file=/Users/sam/Desktop/hehe/ns3-dcn/ns3/src/internet/model/tcp-socket-base.cc, line=1760  
NS_FATAL, terminating  
libc++abi: terminating  
Command 'build/scratch/ns3.46.1-DCN-default --layers=2 --k=4 --torCount=4 --aggCount=2 --serversPerTor=6  
--linkRate=500Mbps --serverToTorRate=500Mbps --torToAggRate=750Mbps --aggToCoreRate=750Mbps --  
linkDelay=100us --tcp=ns3::TcpDctcp --queue=ns3::RedQueueDisc --redMinThresholdPct=15.0 --  
redMaxThresholdPct=15.0 --load=80.0 --workload=Google_AllRPC --  
runTag=L2_k4_t4_a2_s6_d100us_rsta500Mbps_rta750Mbps_rac750Mbps_tcpTcpDctcp_qRedQueueDisc_load80_wGoogle_A  
llRPC_redmin15_redmax15' died with <Signals.SIGABRT: 6>.
```

# Appendix

## Queue Sizing and Simulation Time

$$\text{maxRTT} = 2 \times \text{maxHostHops} \times \text{linkDelay}$$

$$\text{QueueCapacity}_{\text{bytes}} = \frac{\text{linkRate} \times \text{maxRTT}}{8}$$

$$\text{simTime} = 10 \times \frac{\text{maxRTT}}{\text{referenceRTT}}$$



Sam Lee  
EPFL  
sam.lee@epfl.ch

[samlee.ch](http://samlee.ch)